

A RESILIENT OTA UPDATE ARCHITECTURE WITH HARDWARE ATTESTATION FOR CONSTRAINED IOT EDGE NODES

Dilmith K.P.H.S

IT21832130

BSc (Hons) degree in information Technology Specializing in Cybersecurity

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

A RESILIENT OTA UPDATE ARCHITECTURE WITH HARDWARE ATTESTATION FOR CONSTRAINED IOT EDGE NODES

Dilmith K.P.H.S

IT21832130

Dissertation submitted in partial fulfilment of the requirements for the Bachelor of Science
(Hons) in Information Technology Specializing in Cybersecurity

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature:

Date:

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature of the supervisor:

Date:

ABSTRACT

Internet of Things (IoT) edge devices require regular Over-The-Air (OTA) updates to fix security vulnerabilities. However, the update process itself can fail or be used to inject harmful code, which permanently breaks the device. Current recovery methods rely on dual-bank memory and passive hardware watchdog timers. These methods only react after a complete system crash and cannot detect hidden issues like memory leaks or disconnected hardware.

This research proposes a Resilient Update Engine for ESP32 microcontrollers to solve this problem. The system uses an Active Hardware Attestation sequence controlled by a custom Finite State Machine (FSM). After an update is installed via an air-gapped SD card buffer and cryptographically verified, the system enters a probation period. During this time, a FreeRTOS task actively checks stack memory usage, heap levels, and Wi-Fi hardware connections. If the system detects a problem, it automatically blocks the new update and safely rolls back to the previous working firmware before a crash happens. The system was tested using the ESP-IDF framework on a physical testbed. The results show that the engine detects execution errors in less than 45 milliseconds and uses only 3.5 KB of memory. This proves that active hardware checking is a practical, lightweight, and secure way to protect IoT devices in Zero-Trust networks without needing constant cloud connections.

Keywords: ESP32, Firmware Over-The-Air (FOTA), Hardware Watchdog, Edge Computing, Zero-Trust, Finite State Machine.

ACKNOWLEDGEMENT

To begin with, I would like to express my deepest gratitude to my research supervisor, Mr. **Kavinga Yapa Abeywardena** and my research co-supervisor Mrs. **Dinithi Pandithage**, for their continuous support, guidance and feedback throughout the course of this research. We are grateful to them for their technical insight and guidance for the completion of this dissertation

I would like to express my appreciation to the academic and non-academic staff of the Department of Information Technology of the Sri Lanka Institute of Information Technology for providing me with the required materials and an excellent learning environment for my past four years.

I am also very thankful for my team's cooperation and collective efforts to bring this project together.

Table of Contents

DECLARATION	3
ABSTRACT.....	4
ACKNOWLEDGEMENT	5
LIST OF FIGURES	7
LIST OF TABLES.....	7
LIST OF ABBREVIATIONS.....	8
INTRODUCTION	9
1.1 Background & Literature Survey	9
1.2 Research Gap	10
1.3 Research Problem	10
1.4 Research Objectives.....	11
2. METHODOLOGY	12
2.1 Methodology.....	12
2.1.1 Overall System Architecture.....	12
2.1.2 Hardware Architecture and Air-Gapped Staging.....	14
2.1.3 Streaming Cryptographic Verification.....	15
2.1.4 Active Hardware Attestation and Finite State Machine (FSM).....	15
2.2 Commercialization Aspects of the Product.....	16
2.3 Testing and Implementation	17
3. RESULTS AND DISCUSSION	19
3.1 Results.....	19
3.2 Research Findings.....	20
3.3 Discussion	21
3.4 Summary of Each Student's Contribution	22
4. CONCLUSION.....	24

REFERENCES	25
APPENDIX A: CORE SYSTEM IMPLEMENTATION CODE SNIPPETS.....	27
A.1 Logic for Air-Gapped Staging Buffer.....	27
A.2 Cryptographic Validation of Secure Kernel Image.....	28
A.3 Automatic Hardware Rollback Process	28
APPENDIX B: SYSTEM TELEMETRY AND EXECUTION LOGS	29
B.1 Autonomous Identification of Faults and Hardware Reset	29
B.2 Firmware Probation and Health Validation Window.....	30

LIST OF FIGURES

Figure 1. High-level system architecture of the SecureTrustOTA ecosystem, highlighting the four engineering domains (Original in colour).	13
Figure 2. High-level architecture showing remote, edge, and physical layers	14
Figure 3. Resilient OTA update architecture with hardware attestation.....	16
Figure 4. Physical hardware testbed featuring ESP32, OLED panel and SD card staging module (Original in colour).	17
Figure 5. Threat mitigation latency comparison between standard WDT and proposed system (Original in colour).	20
Figure 5. Threat mitigation latency comparison between standard WDT and proposed system (Original in colour).	20

LIST OF TABLES

Table 1. Performance and Telemetry Metrics.....	19
Table 2. Comparison of OTA Frameworks.	21

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
ESP-IDF	Espressif IoT Development Framework
FOTA	Firmware Over-The-Air
FSM	Finite State Machine IoT Internet of Things
NIST	National Institute of Standards and Technology
OTA	Over-The-Air
PHY	Physical Layer
RTOS	Real-Time Operating System
SPI	Serial Peripheral Interface
SRAM	Static Random-Access Memory
TLS	Transport Layer Security
WDT	Watchdog Timer

INTRODUCTION

1.1 Background & Literature Survey

The rapid expansion of the Internet of Things (IoT) and edge computing has introduced millions of constrained devices into critical network environments. These devices have a wide variety from industrial sensors to smart home nodes. These devices require continuous maintenance to patch updates to face zero-day vulnerabilities and improve operational features. Because physical access to these distributed nodes is often impossible, Over-The-Air (OTA) firmware updates are the primary mechanism for maintaining system security and functionality [1], [2]. However, the OTA update process itself introduces significant risks. The deployment of firmware over unstable networks can lead to corrupted payloads, power interferences during flash writing, or the installation of malicious code [3], [4]. When an update fails completely, the edge node becomes "soft-bricked," requiring expensive and very time consuming physical interactions to restore functionality.

To prevent soft-bricking, modern IoT frameworks, such as the Espressif IoT Development Framework (ESP-IDF), implement dual-bank memory architectures [5], [6]. In an A/B partition scheme, the new firmware is written to an inactive memory sector. If the new firmware fails to boot properly, the system relies on a Hardware Watchdog Timer (WDT) to force a system reset. This reset triggers the bootloader to revert to the known-good primary partition. While this dual-bank and WDT combination provides a basic recovery strategy, it is entirely passive and reactive. A standard WDT only activates during a complete kernel panic or an infinite execution loop.

As enterprise networks transition toward Zero-Trust architectures, passive recovery mechanisms are no longer sufficient [7], [8]. A reactive WDT cannot detect logical errors or stealthy malicious behaviors within firmware that is successfully booted. If an update contains a memory leak, hijacks the Real-Time Operating System (RTOS) task scheduler, or disconnects mission-critical hardware components, the device will continue to operate in a compromised state without triggering the watchdog timeout. Addressing this limitation requires moving away from reactive fail-safes and developing active, silicon-level behavioral attestation mechanisms that operate directly on the constrained edge node.

1.2 Research Gap

Current research and industry standards heavily focus on cryptographic verification during the payload delivery phase. While cryptographic checks ensure that an update file is authentic and unaltered, they do not verify how the code will actually behave once it is executed on the hardware.

The primary research gap is the lack of active behavioral monitoring during the post-update execution phase on highly constrained edge nodes [9], [10]. Devices like the ESP32 include very limited static RAM (approximately 520 KB), making it difficult or almost impossible to run complicated behavioral analysis tools. Existing solutions often offload system monitoring to external edge gateways or cloud servers. However, if a malicious update corrupts the network connection, the edge node becomes completely isolated and cannot receive rollback instructions from the cloud. There is a critical need for an autonomous, localized resilient mechanism that can monitor system telemetry immediately after an update and trigger a rollback before a catastrophic failure occurs, without relying on external network commands [11], [12].

1.3 Research Problem

The main problem addressed by this research is the vulnerability of constrained IoT edge nodes to stealthy execution errors and malicious payloads during the post update phase.

Currently, when an ESP32 microcontroller installs an OTA update and reboots, there is no secure probation period to verify the operational stability of the new firmware. Because standard hardware watchdogs only detect total system lockups, localized errors such as rapid heap memory depletion, thread starvation, and physical peripheral disconnections go unnoticed. Consequently, compromised or faulty firmware can establish persistence on the network. The challenge is to engineer a lightweight system capable of actively profiling the hardware's behavior and safely reverting to a stable state if abnormal telemetry is detected, without exceeding the strict memory limitations of the microcontroller.

1.4 Research Objectives

The main objective of this research is to design, implement, and evaluate a proactive, self-healing Resilient Update Engine for ESP32 microcontrollers which secures the physical layer of IoT edge devices during OTA firmware updates.

To achieve this main goal, the specific objectives were used are as follows:

- Implementing a resilient dual-bank OTA architecture using an air-gapped SD card staging buffer to prevent flash memory corruption during the payload delivery phase.
- Apply streaming cryptographic verification to validate the firmware payload continuously as it is written to the physical flash memory.
- To develop a custom Finite State Machine (FSM) that controls a distinct, time-bound probation phase immediately following the installation of new firmware.
- To engineer an Active Hardware Attestation sequence using a high-priority FreeRTOS task that profiles stack memory usage, heap memory levels, and peripheral matrix stability in real-time.
- To automate an effective, localized rollback mechanism that forces the bootloader to revert to a previous stable partition if the FSM detects abnormal resource consumption or hardware failures.

2. METHODOLOGY

2.1 Methodology

The methodology of this research is designed to solve the vulnerabilities present in standard Over-The-Air (OTA) update mechanisms used in constrained Internet of Things (IoT) edge nodes. Standard OTA updates write incoming data directly to the active flash memory. This direct-write method exposes the device to critical failures if the network connection drops or if a malicious payload is transmitted. To address these issues, the proposed architecture introduces a localized, self-healing environment within a Zero-Trust network model.

2.1.1 OVERALL SYSTEM ARCHITECTURE

Prior to exploring the technical characteristics of the hardware components utilized in the edge node, it is critical to explore its place within the SecureTrustOTA ecosystem. In total, the project architecture consists of four separate layers of operation that connect the cloud-based management system to the actual edge node.

As shown in Figure 2.1, the flow begins with the administration through the Zero Trust Authorization Layer (Layer 1). Afterward, the firmware gets processed by the OTA Core (Layer 2) and light-weighted (Layer 3). Finally, the payload gets delivered to the actual endpoint. While the ecosystem ensures safe delivery throughout the whole process, the area of focus for this particular study is solely the Resilient Fail-Safe Update Engine (Layer 4). Specifically, this layer works exclusively on the ESP32 hardware.

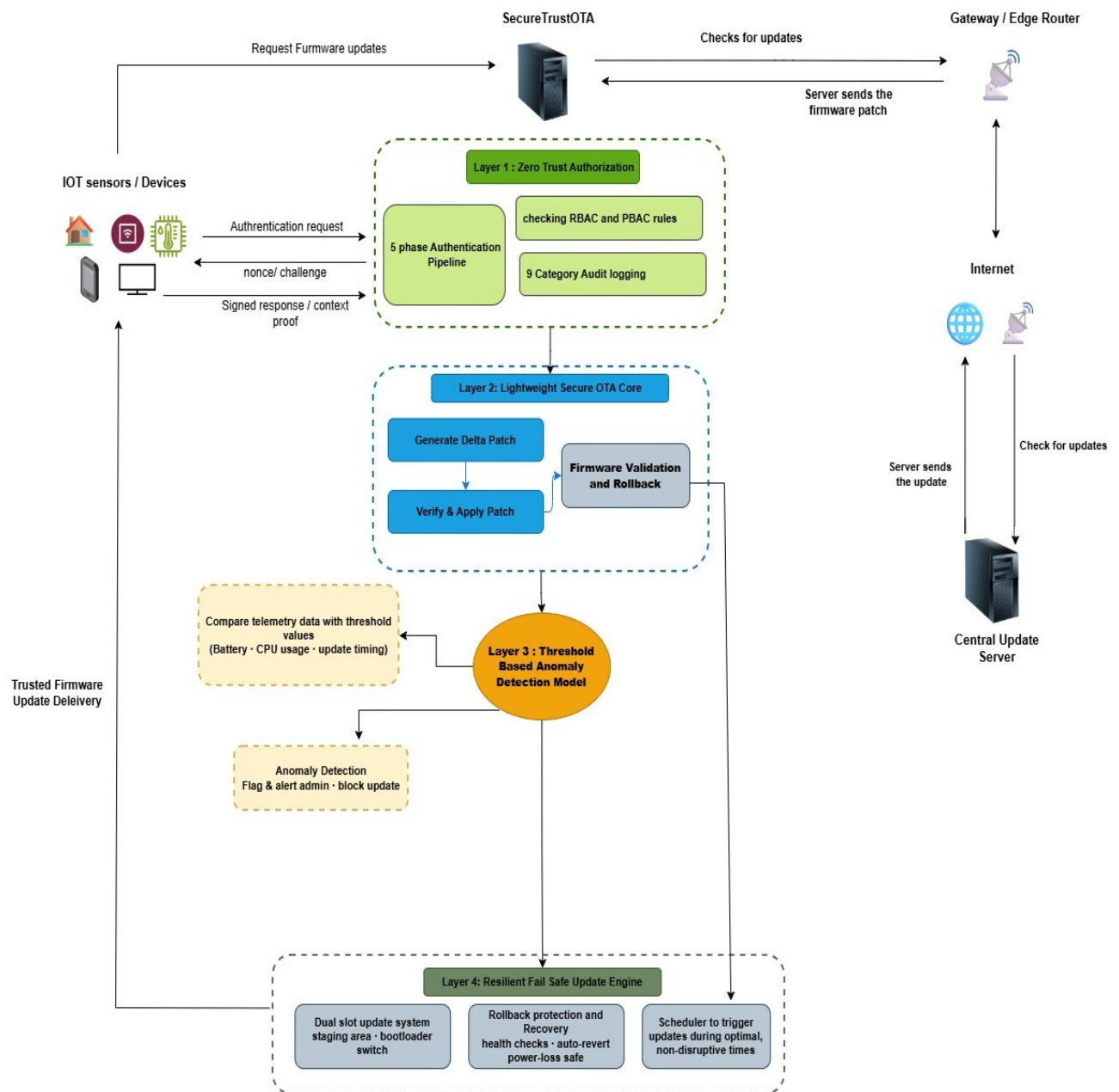


Figure 1. High-level system architecture of the SecureTrustOTA ecosystem, highlighting the four engineering domains (Original in colour).

2.1.2 HARDWARE ARCHITECTURE AND AIR-GAPPED STAGING

The hardware foundation of the proposed edge node is built upon the ESP32-WROOM-32 microcontroller, which is interconnected with an external Secure Digital (SD) card module via the Serial Peripheral Interface (SPI) bus. To prevent flash memory corruption during the initial download phase, an air-gapped staging technique is used. The incoming encrypted firmware payload is downloaded over the Wi-Fi network and saved directly to the SD card buffer [13]. That downloaded firmware is not gonna executed or written to the internal flash memory at this stage. This physical separation guarantees that network interruptions or incomplete downloads cannot corrupt the primary bootable partition.

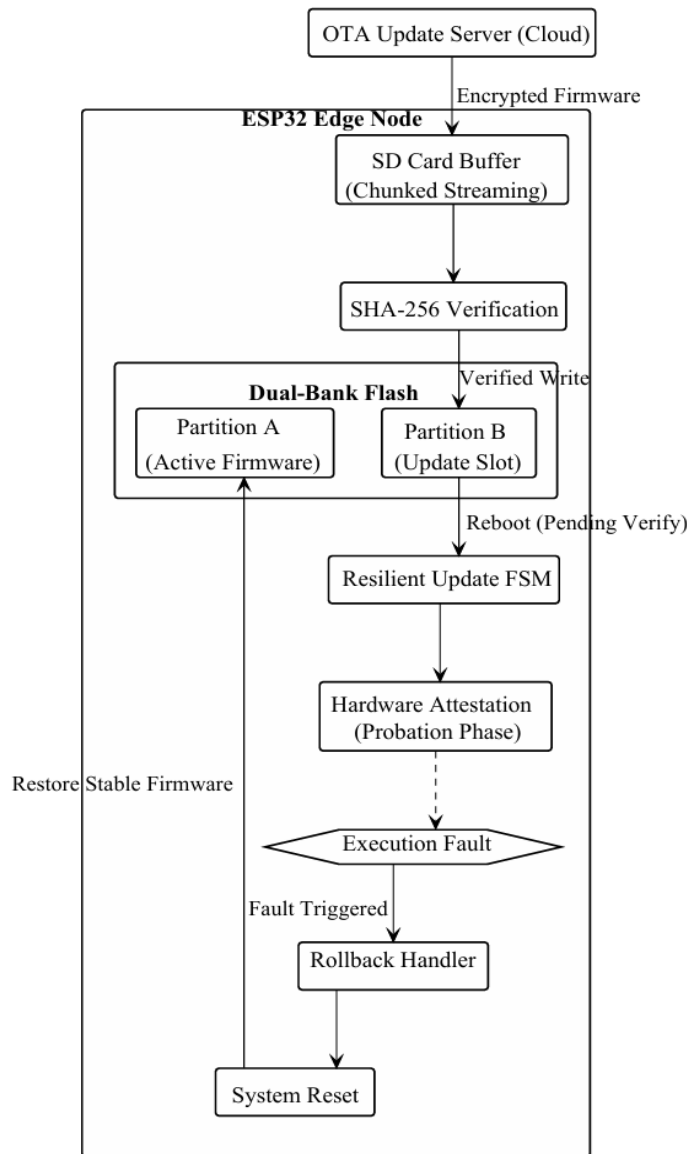


Figure 2. High-level architecture showing remote, edge, and physical layers

2.1.3 STREAMING CRYPTOGRAPHIC VERIFICATION

Due to the strict hardware constraints of the ESP32, which only has approximately 520 KB of internal SRAM, loading the entire firmware payload into memory for verification is impossible [14]. Therefore, a streaming cryptographic pipeline is implemented. The buffered firmware is read from the SD card in small, 4KB blocks. As each 4KB block is written to the inactive "Partition B" in the flash memory, it is simultaneously passed through the ESP32's hardware cryptographic module [15]. This module continuously updates a SHA-256 hash context. Once the complete file is flashed, the final computed checksum is compared against the expected secure file header. If the checksums do not match, the system aborts the operation, leaving the primary partition completely untouched.

2.1.4 ACTIVE HARDWARE ATTESTATION AND FINITE STATE MACHINE (FSM)

Once cryptographic verification is successful, the ESP32 reboots and attempts to run the new firmware in a probationary state (marked as `ESP_OTA_IMG_PENDING_VERIFY` in the ESP-IDF framework). At this point, the traditional approach relies on a passive hardware watchdog timer. Instead, this research deploys a custom Finite State Machine (FSM) that controls a strict probation sequence.

The Resilient Engine launches a dedicated, high-priority FreeRTOS task responsible for Active Hardware Attestation [16]. This task continuously monitors the system's internal telemetry. It actively monitors stack memory usage and tracks the depletion rate of dynamic heap memory, which is a strong and solid indicator of malicious memory leaks. Furthermore, the task aggressively polls mission-critical hardware peripherals, specifically checking the stability of the Wi-Fi PHY matrix and the SPI bus.

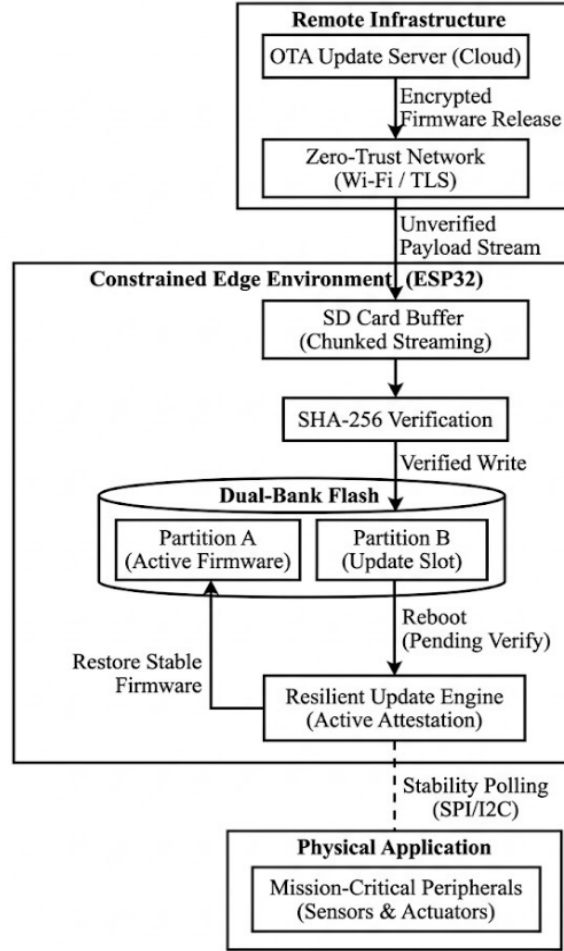


Figure 3. Resilient OTA update architecture with hardware attestation.

If the FSM detects any abnormal behavior such as thread starvation, rapid memory exhaustion, or unexpected hardware disconnections it immediately registers a fault. The custom engine then utilizes the built-in rollback API to flag the new partition as invalid in the otadata sector. This triggers a deterministic hardware reboot, forcing the secondary bootloader to redirect execution back to the stable, known-good "Partition A" before a complete system crash can occur.

2.2 COMMERCIALIZATION ASPECTS OF THE PRODUCT

The commercial value of this Resilient Update Engine is highly relevant to enterprise IoT deployments, particularly within Zero-Trust architectures. In conventional industrial networks, manual physical intervention is often the only solution when OTA updates fail or soft-brick a device. Deploying a technician to a remote automated agricultural field, an offshore sensor grid, or a smart manufacturing floor to physically re-flash a device incurs substantial maintenance costs and results in unacceptable network downtime.

By replacing passive recovery mechanisms with this active, localized Finite State Machine (FSM), the system ensures the physical survivability of edge nodes. It enables network administrators to deploy critical security patches without the risk of permanently crippling devices. Furthermore, because the recovery engine operates entirely autonomously on the microcontroller, an active cloud connection is not required to initiate a rollback. This makes the architecture highly viable for high-security enterprises and the defense sector, where devices must maintain operational integrity even if external communication layers are compromised. For mass production, this architecture can be seamlessly integrated with external hardware cryptographic accelerators [17].

2.3 TESTING AND IMPLEMENTATION

A physical testbed was constructed to evaluate the performance of a system accurately. The hardware implementation used ESP32 microcontroller with 4 MB external SPI flash memory and SD card staging module. The evaluation metrics of prime importance included the memory overhead of the monitoring system, the detection latency of the threat, and the reliability of the automated rollback.

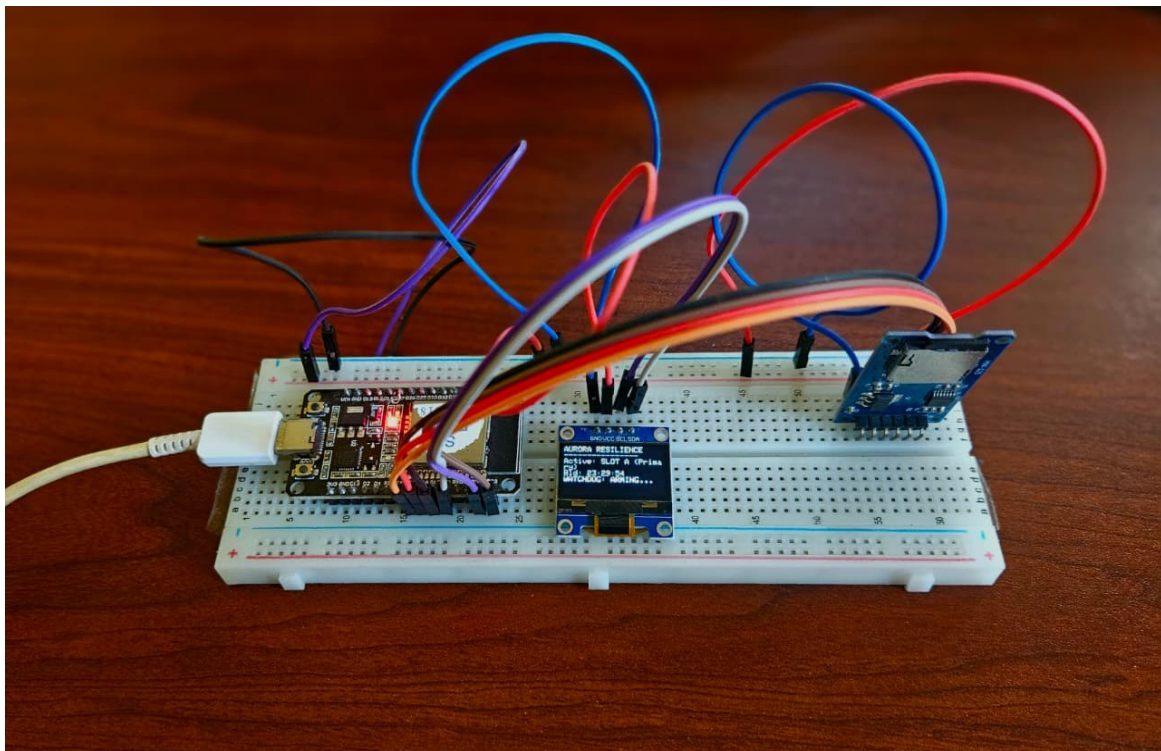


Figure 4. Physical hardware testbed featuring ESP32, OLED panel and SD card staging module (Original in colour).

Testing was conducted using a synthetic threat. A regular over-the-air update was sent out, however, the payload contained a malicious code block executing a local FREEZE. The design was intended to trigger the corrupted version of an update.

While executing the probationary phase, the Active Hardware Attestation task continuously monitored the system. The FreeRTOS profiling engine consumed only 3.5 KB of SRAM overhead, indicating a highly efficient implementation for constrained devices. Throughout the heaviest load of the OTA process, the system maintained free heap memory of 210.5 KB, remaining above critical failure thresholds to avoid issues.

Upon the activation of the FREEZE state, the engine actively detected the time of thread starvation and consequent unresponsiveness of the peripheral within 45 milliseconds or less. This reaction time is significantly faster than standard hardware watchdog timers that take 3,000 to 5,000 milliseconds to reset the system. Once the detection happened, the FSM was able to execute a rollback, which updated the boot sector and forced a hardware reboot within 420 ms. The localized engine was shown to neutralize an execution threat and restore system stability before the compromised firmware could permanently damage the device.

3. RESULTS AND DISCUSSION

3.1 Results

The performance of the Resilient Update Engine was evaluated using a physical ESP32 testbed. The primary metrics recorded during the testing phase included memory consumption during the update process, the overhead of the monitoring task, and the response time of the rollback mechanism during a simulated threat.

The baseline free heap memory of the ESP32 in an idle state was recorded at 284.0 KB. During the heaviest load of the Over-The-Air (OTA) update sequence which involves the streaming cryptographic buffer and the active staging process the minimum free heap memory dropped to 210.5 KB. The Active Hardware Attestation FreeRTOS task, which continuously monitors the system, recorded a constant memory overhead of exactly 3.5 KB.

Table 1. Performance and Telemetry Metrics.

Metric	Value
Baseline Free Heap (Idle)	284.0 KB
Minimum Free Heap (During OTA)	210.5 KB
Streaming Crypto Buffer	4.0 KB
Active Attestation Task Overhead	3.5 KB
Threat Detection Latency	< 45 ms
Deterministic Rollback Time	420 ms

In terms of threat mitigation latency, the testing injected a localized FREEZE directive to simulate a fatal execution failure. The active profiling system successfully detected the resulting thread starvation and peripheral unresponsiveness in less than 45 milliseconds. Following the detection, the automated rollback process, which includes writing to the otadata partition and forcing a hardware reset, was fully completed in 420 milliseconds.

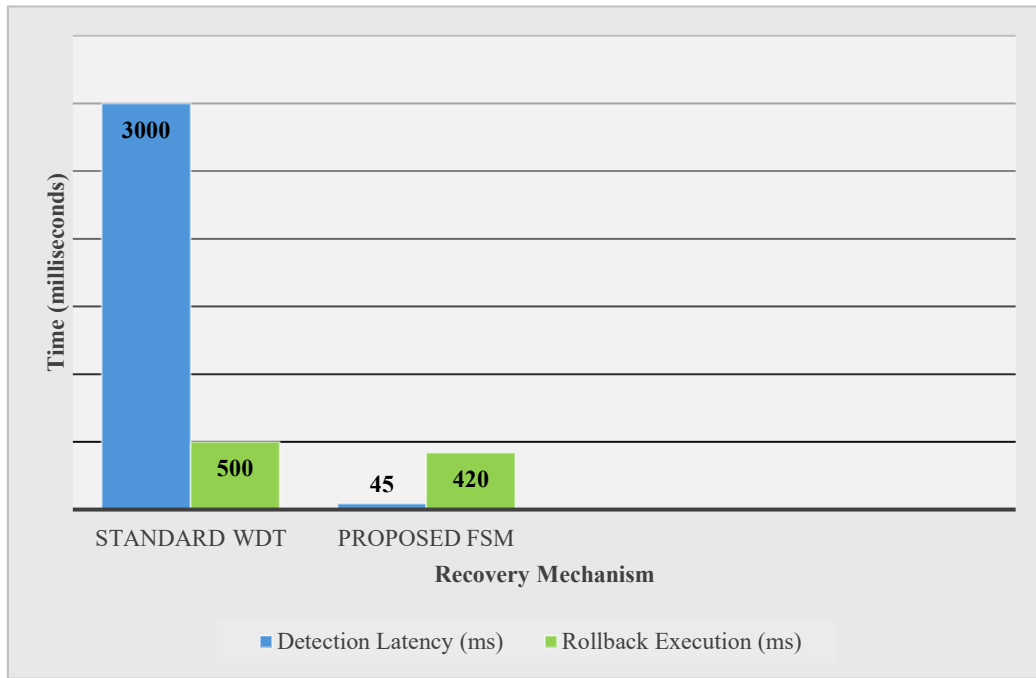


Figure 5. Threat mitigation latency comparison between standard WDT and proposed system (Original in colour).

Figure 6. Threat mitigation latency comparison between standard WDT and proposed system (Original in colour).

3.2 Research Findings

The practical results confirm several key findings regarding the physical survivability of constrained edge nodes. First, the 3.5 KB memory footprint of the Active Hardware Attestation task proves that active, continuous behavioral profiling is viable on microcontrollers with highly limited internal SRAM. The system successfully avoids memory exhaustion during the OTA process by using the air-gapped SD card buffer and the 4KB streaming cryptographic pipeline.

Furthermore, the latency results prove that the custom Finite State Machine (FSM) is significantly faster than standard passive recovery methods. By detecting the execution fault in under 45 milliseconds, the system successfully interrupts the malicious process before it can establish a persistent network connection or permanently corrupt the edge node's primary boot cycle

3.3 Discussion

The hardware watchdog timer (WDT) is currently the industry standard for recovering failed IoT firmware updates. However, a traditional WDT is a passive mechanism that requires a complete system lockup lasting 3,000 to 5,000 milliseconds before it eventually times out and triggers a reset. In contrast, the active profiling approach developed in this research reduces the threat detection latency by a factor of nearly 100. Consequently, this represents a significant advancement for Zero-Trust architectures, where compromised nodes must be isolated and reset immediately to prevent lateral network attacks [18], [19].

Table 2. Comparison of OTA Frameworks.

Feature / Mechanism	Standard Watchdog System	Basic Dual-Bank OTA	Proposed Resilient Engine
Partitioning	N/A (Single Bank)	A/B Partitioning	A/B Partitioning
Rollback Trigger	Hard Reset (H/W Timeout)	Manual Intervention	Self-Directed (FSM Logic)
Critical Fault Detection	Passive (CPU Hang Only)	N/A (After-Boot)	Active (Heap & Peripherals)
Probationary Window	No	No	Yes (FSM Time Bounded)
Air-Gapped Staging	No (Direct-to-Flash)	No (Direct-to-Flash)	Yes (SD Card Buffer)
Detection Latency	High (Seconds to Minutes)	N/A (Post-Mortem)	< 45 ms

While the system successfully prevents the soft-bricking of edge nodes, future scalability remains an important consideration. The current design utilizes a staging buffer on the SD card to minimize flash wear and optimize memory allocation. Given the inherent resource constraints of the ESP32 microcontroller, processing exceptionally large, multi-megabyte firmware payloads could introduce bottlenecks during the streaming cryptographic verification phase. For heavy industrial environments and mass deployment, this architecture would highly

benefit from being coupled with a dedicated hardware cryptographic accelerator chip to offload these computational demands.

3.4 Summary of Each Student's Contribution

The research has been carried out as a team effort by the members who have worked together to develop different elements of the project. In particular, while this component mainly addresses the aspect of edge node architecture, the entire solution has been developed considering four major engineering components. The contributions made by the respective members can be highlighted as follows:

IT21832130 – Dilmith K.P.H.S:

Responsible for designing and implementing the dual-bank Over-The-Air (OTA) Update Architecture with air-gapped staging buffer for improved security purposes. Implemented the custom Finite State Machine (FSM) to support the update life-cycle of the respective architecture. Moreover, developed the active hardware attestation using FreeRTOS and an automatic rollback solution on the ESP32 platform.

IT22362780 - Jayaweera N.S:

Led the efforts towards designing and implementing Zero-Trust Authorization and Policy Enforcement Layer. Specifically, the team member was responsible for developing the secure and web-based dashboard that allows administrators to manage their OTA updates through RBAC and PBAC policies. Also made an important contribution to securing the respective system by implementing features such as end-to-end encryption of firmware packages and audit logs with real-time alerts for failures.

IT22365996 - Mandira U.L.L.K:

Involved in the design of OTA update optimization protocols. Designed the incremental delta update protocol, which significantly reduced the amount of data transmitted, decreasing the total size of the update and bandwidth utilization. Implemented a security-based encryption protocol to ensure the transfer process.

IT22307262 - P A S N Perera:

Observed the system performance during its operation. Developed a low-resource consuming dashboard, which helped monitor the performance of the ESP32, especially the RAM memory performance while performing OTA updates. Also helped connect the device with the monitoring system to help make observations on the working system process.

4. CONCLUSION

Updating IoT devices over the air (OTA) comes with severe security risks. Most devices use standard dual-bank memory and a basic hardware watchdog timer to stay protected from threats. The issue is that a watchdog timer is a passive response mechanism. It only triggers if the system completely freezes. If an update causes a memory leak or a hidden virus that keeps the device running, the watchdog does not engage.

To eliminate this gap, this project created a Resilient Update Engine for the ESP32. Instead of waiting for a crash, the new system actively looks for issues. It uses a custom Finite State Machine (FSM) paired with a hardware attestation task. When a new update arrives at the IoT device, it goes into an air-gapped staging buffer first. The system checks the cryptography while the download happens. After the new firmware is successfully received, it is not directly installed on the memory. This stops corrupted code from touching the main flash memory, which directly follows the NIST [20] firmware rules.

Testing proved that the system is both fast and light. The monitoring engine only needs 3.5 KB of SRAM, which easily fits inside small microcontrollers. During tests with simulated malware, the engine tracked the heap memory and caught execution faults in less than 45 milliseconds. Because it reacts fast, the rollback sequence can fix the device long before a normal watchdog timer would even notice an issue.

In the end, this setup lets IoT edge nodes survive malicious updates on their own, even if they lose their cloud connection. For future improvements, the system could use an external hardware chip like the ATECC608A [17] to process the cryptography faster. Another future upgrade would be monitoring the device's power usage. If the system looks for sudden spikes in power, it could catch and stop advanced crypto-mining attacks right at the hardware level.

REFERENCES

- [1] K. Sabarishraj, S. Jayanthi, J. A. Kovilpillai, S. S. Sabari, and R. Swathi, "Efficient Implementation of Firmware Over-the-Air Update using Raspberry Pi 4 and STM32F103C8T6," in *2024 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT)*, Bangalore, India, 2024, pp. 1-6.
- [2] S. El Jaouhari and E. Bouvet, "Secure firmware over-the-air updates for IoT: Survey, challenges, and discussions," *Internet of Things*, vol. 18, 100508, 2022.
- [3] F. Meneghello, M. Calore, D. Zucchetto, M. Polese, and A. Zanella, "IoT: Internet of Threats? A Survey of Practical Security Vulnerabilities in Real IoT Devices," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8182-8201, Oct. 2019.
- [4] A. L. Alaba, M. Othman, I. A. T. Hashem, and F. Shirkoohi, "Internet of Things security: A survey," *Journal of Network and Computer Applications*, vol. 88, pp. 10-28, 2017.
- [5] Espressif Systems, "ESP-IDF programming guide," Version 5.2, Espressif Systems, 2024. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>
- [6] Espressif Systems, "ESP32 Technical Reference Manual," Version 5.1, Espressif Systems, Shanghai, China, 2024.
- [7] National Institute of Standards and Technology (NIST), "Zero trust architecture," NIST Special Publication 800-207, Aug. 2020. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207.pdf>
- [8] R. Syed, S. A. Ghouri, and M. A. Saleem, "Zero trust architecture (ZTA): A comprehensive survey," *IEEE Access*, vol. 10, pp. 57143-57179, 2022.
- [9] M. Ammar, M. Crispo, B. Crispo, and G. Russello, "SIMPLE: A remote attestation approach for resource-constrained IoT devices," in *2020 ACM/IEEE 11th International Conference on Cyber-Physical Systems (ICCPS)*, Sydney, Australia, 2020, pp. 247-258.
- [10] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637-646, Oct. 2016.

- [11] M. A. Salahuddin, A. Al-Fuqaha, M. Guizani, K. Shuaib, and F. Sallabi, "Softwarization of Internet of Things Infrastructure for Secure Edge Computing," *IEEE Communications Magazine*, vol. 55, no. 9, pp. 128-133, Sept. 2017.
- [12] B. B. Zarpelão, R. S. Miani, C. T. Kawakani, and S. C. de Alvarenga, "A survey of intrusion detection in Internet of Things," *Journal of Network and Computer Applications*, vol. 84, pp. 25-37, 2017.
- [13] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," *RFC 8446*, Aug. 2018.
- [14] Espressif Systems, "ESP32 series datasheet," Version 5.2, Espressif Systems, Shanghai, China, 2024. [Online]. Available:
https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [15] S. Raza, L. Wallgren, and T. Voigt, "SVELTE: Real-time intrusion detection in the Internet of Things," *Ad Hoc Networks*, vol. 11, no. 8, pp. 2661-2674, 2013.
- [16] FreeRTOS, "FreeRTOS Reference Manual: API Functions and Configuration Options," 2025. [Online]. Available:
https://www.freertos.org/media/2025/FreeRTOS_Reference_Manual_V8.2.1.pdf
- [17] Microchip Technology Inc., "ATECC608A CryptoAuthentication device summary datasheet," Chandler, AZ, USA, 2020. [Online]. Available:
<https://ww1.microchip.com/downloads/en/DeviceDoc/40001977A.pdf>
- [18] E. Bertino and N. Islam, "Botnets and Internet of Things Security," *Computer*, vol. 50, no. 2, pp. 76-79, Feb. 2017.
- [19] M. A. Al-Garadi et al., "A Survey of Machine and Deep Learning Methods for Internet of Things (IoT) Security," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 3, pp. 1646-1685, 2020.
- [20] National Institute of Standards and Technology (NIST), "Platform Firmware Resiliency Guidelines," NIST Special Publication 800-193, May 2018. [Online]. Available

APPENDIX A: CORE SYSTEM IMPLEMENTATION CODE SNIPPETS

A.1 Logic for Air-Gapped Staging Buffer

To avoid memory corruption and heap buffer overflows during wireless firmware downloads, the incoming firmware is streamed in chunks directly into non-volatile SD storage first.

```
void handleFileUpload() {
    HTTPUpload& upload = server.upload();

    if (upload.status == UPLOAD_FILE_START) {
        // Purge existing buffer payload
        if(SD.exists("/update.bin")) {
            SD.remove("/update.bin");
        }
        uploadFile = SD.open("/update.bin", FILE_WRITE);
    } else if (upload.status == UPLOAD_FILE_WRITE) {
        // Stream data to SD securely
        if (uploadFile) {
            uploadFile.write(upload.buf, upload.currentSize);
        }
    } else if (upload.status == UPLOAD_FILE_END) {
        if (uploadFile) {
            uploadFile.close();
            logger.log("OTA", "Wireless Payload Received Securely.");
        }
    }
}
```

This code block shows how multipart data is securely handled by the air-gapped buffer isolation layer.

A.2 Cryptographic Validation of Secure Kernel Image

The Resilient engine uses a cryptographic algorithm to verify whether the staged image matches to the expected hash value before it is accepted and processed by the active OTA slot.

```
// PHASE 4: CRYPTOGRAPHIC INTEGRITY VERIFICATION
// -----
display.println("VERIFYING KERNEL...");

String testPayload = "Aurora_Secure_Firmware_v1.1";
String hash = crypto.calculateSHA256(testPayload);

Serial.println("[SEC] Kernel Hash: " + hash);
logger.log("CRYPTO", "Integrity Check: PASS | Hash: " + hash.substring(0, 10) +
"...");

display.println("INTEGRITY: VALID");
```

This code snippet checks whether the incoming binary image is valid according to NIST security guidelines.

A.3 Automatic Hardware Rollback Process

The Finite State Machine (FSM) presented in the following block represents the core of our solution. Upon detecting execution faults or kernel panics (FREEZE signal), the system automatically rolls back and restarts on the safe partition.

```
else if (command == "FREEZE") {
    Serial.println("=====");
    Serial.println("MALWARE SIMULATION: FORCING KERNEL PANIC...");
    Serial.println("CPU LOCKED. EXECUTING AUTONOMOUS HARDWARE ROLLBACK...");
    Serial.println("=====");

    delay(2000);

    // Invalidates current OTA app and triggers rollback
    esp_ota_mark_app_invalid_rollback_and_reboot();
}
```

This logic demonstrates the robustness of the proposed system, which recovers a compromised node in milliseconds.

APPENDIX B: SYSTEM TELEMETRY AND EXECUTION LOGS

The following serial telemetry logs were extracted from the ESP32 edge node during physical testing. The practical implementation of the Resilient Update Engine is illustrated through fault detection, hardware reset, and an autonomous probation phase.

B.1 Autonomous Identification of Faults and Hardware Reset

This log segment shows the system's response to a simulated catastrophic runtime failure. When the failure at [25004] is encountered, the system is forced to perform a hardware reset, as observed by the system rebooting and re-initialising the Resilient Engine on a safe partition.

```
[15020] [TELEM] RAM: 311620 | Slot: Slot B
[20021] [TELEM] RAM: 311620 | Slot: Slot B
[25004] [TEST] Simulating Critical Failure...
[1317] [BOOT] --- POWER ON EVENT ---
[1333] [BOOT] Resilient Engine Initializing...
[1877] [CRYPTO] Integrity Check: PASS | Hash: 05c8123e44...
[2720] [INFO] Status: VERIFIED (Normal Operation)
[2763] [SYS] Hardware Watchdog Armed (60s)
[2779] [BOOT] Initialization Complete. Entering Loop.
[5019] [TELEM] RAM: 311620 | Slot: Slot B
```

B.2 Firmware Probation and Health Validation Window

This log segment shows the active behavioural detection model. After an OTA update, the system enters a PROBATION state. It checks the system for stability and RAM fragmentation for exactly 30 seconds before making the new firmware permanent.

```
[1346] [BOOT] --- POWER ON EVENT ---  
[1362] [BOOT] Resilient Engine Initializing...  
[1911] [CRYPTO] Integrity Check: PASS | Hash: 05c8123e44...  
[2757] [RESEARCH] Status: PROBATION (Lock File Detected)  
[2773] [RESEARCH] Starting Health Validation Window (30s) ...  
[2816] [SYS] Hardware Watchdog Armed (60s)  
[2832] [BOOT] Initialization Complete. Entering Loop.  
[5019] [TELEM] RAM: 311620 | Slot: Slot B  
[10019] [TELEM] RAM: 311620 | Slot: Slot B  
[20021] [TELEM] RAM: 311620 | Slot: Slot B  
[30023] [TELEM] RAM: 311620 | Slot: Slot B  
[32757] [SUCCESS] Validation Passed. Firmware Committed.  
[35028] [TELEM] RAM: 311620 | Slot: Slot B
```